

Week 11 In-Class Exercise

Testing, Verification, and Multicore Stabilization

Open-Source Prototype Systems and Applications | 4-Hour Lab

Lab theme: a small multicore system can pass unit tests and still fail after integration. Students debug stale reads, address-contract bugs, lost invalidation pulses, and race-triggered failures using Verilator, trace replay, assertions, and a gem5-level stabilization discussion.

0. Setup Summary

Item	Value
Docker image	amansinhaatnycu/osp:latest
Pre-installed tools assumed	Verilator and gem5
Main command path	make run-baseline, make run-case-b, make run-case-c, make run-case-d, make gem5-producer-consumer
Student deliverable	short debug report, one new trace, one new assertion, and Gem5 producer-consumer connection
Core lesson	stabilization means preserving system-level invariants under realistic integration stress

```
docker run --rm -it -v "$PWD":/work amansinhaatnycu/osp:latest bash
cd /work/week11_osp_lab_package
make clean
make run-baseline
```

1. Learning Objectives

- **Verification boundary:** distinguish module correctness from integrated multicore correctness
- **Invariant thinking:** state correctness as properties: read-after-write, invalidation, full-address matching, deterministic concurrency
- **Trace discipline:** use directed traces to reproduce bugs instead of relying only on random testing
- **Assertion discipline:** encode stabilization rules close to the RTL where violations first appear
- **Debug observability:** use debug pulses, hit signals, and self-checking logs without confusing visibility with correctness
- **Architecture connection:** run a real two-core Gem5 producer-consumer workload and connect software synchronization to RTL-level visibility, address, and ordering contracts

2. Classroom Flow

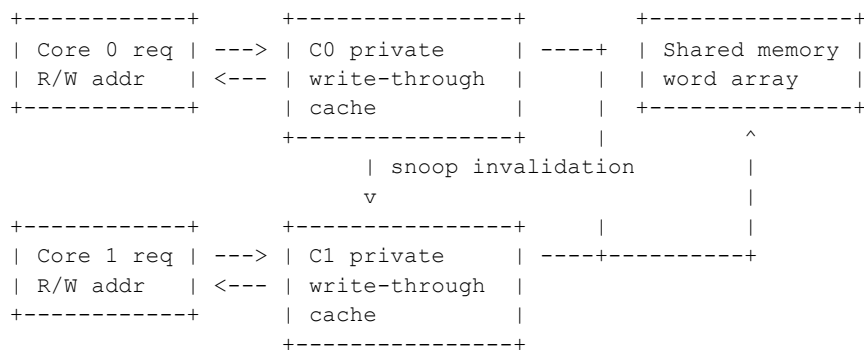
Time	Activity	Instructor goal	Student output
09:10-09:25	System briefing and Docker launch	establish architecture and commands	working terminal and baseline build
09:25-09:40	Baseline run and invariant extraction	show what a stable system looks like	list three invariants from PASS log
09:40-10:00	Investigation Case B	debug from first failing observation	hypothesis and signal evidence
10:00-10:10	Break	reset groups	none
10:10-10:15	Investigation Case C	test address-identity contract	address decode and cache-state explanation
10:15-10:40	Investigation Case D	test same-cycle stabilization	race explanation and assertion idea
10:40-10:50	Break	prepare final stabilization task	none
10:50-11:15	Group stabilization patch or assertion	students patch one hypothesis or add one bounded check	modified trace, assertion, or minimal RTL patch
11:15-11:30	Gem5 producer-consumer run and wrap-up	connect RTL visibility contracts to two-core software synchronization	stats observation and short report conclusion

3. System Under Test

The RTL models a deliberately small but realistic integration problem: two request sources access shared memory through private direct-mapped write-through caches. The goal is not to build a production coherence protocol. The goal is to test whether simple multicore invariants survive integration.

- **Core 0 and Core 1:** independent request ports with valid, write-enable, address, write-data, read-data, ready, and hit signals
- **Private caches:** four-line direct-mapped caches; each line stores valid bit, tag, and data word
- **Shared memory:** single backing array initialized with deterministic values for reproducible checking
- **Snooped invalidation:** a write by one core invalidates the other core if it holds the same line
- **Debug outputs:** invalidation pulses and hit bits expose symptoms but do not replace self-checking correctness
- Lab variants: compile-time parameter selects baseline behavior or an investigation case

4. Architecture Diagram



Ask students: Which signals are local module behavior, and which signals encode system-level contracts? The answer should include full address propagation, invalidation timing, and read-after-write visibility.

5. Command Sheet

```
# inside the extracted package
make clean
make run-baseline
make run-case-b
make run-case-c
make run-case-d
make trace-check
make gem5-producer-consumer
```

Command	Expected result	What to inspect
make run-baseline	PASS	read values, hit bits, and debug pulses
make run-case-b	FAIL before patch	first mismatch and violated invariant
make run-case-c	FAIL before patch	address pair, hit decision, and cache state
make run-case-d	FAIL before patch	same-cycle request evidence
make trace-check	PASS	golden trace semantics independent of RTL
make gem5-producer-consumer	PASS workload output	stats.txt, producer-consumer visibility pattern

6. Exercise A: Baseline Stabilization Run

Goal: prove that students can read a verification log before they try to debug anything. A PASS result is not just a green line; it is evidence that directed traces, random stress, and assertions agree for the baseline.

```
make run-baseline
```

- **Task A1:** identify the first read that fills Core 0 cache and the first read that fills Core 1 cache
- **Task A2:** find the first write that should invalidate the other core
- **Task A3:** explain why the next read by the other core must refetch from memory
- **Task A4:** write down one property that must remain true regardless of cache hit or miss timing

Question	Expected answer
Why is a second read often a hit?	
Why does a write update memory immediately?	
Why must the other cache be invalidated?	
Why use deterministic memory initialization?	

7. Exercise B: Investigation Case B - First Failing Scenario

`make run-case-b`

- Observed symptom: a later read returns a value that no longer matches the testbench golden model
- Student task: identify the first point where the observed value diverges from the expected value; do not begin by editing random RTL lines
- Debug method: compare the write operation, address fields, local cache state, shared memory value, peer-cache state, and the next read result
- Patch rule: make the smallest RTL edit that restores the violated invariant, then rerun both baseline and the failing case
- **Verification lesson:** a cache can appear correct locally but fail as soon as another core shares memory

Invariant clue	What students should compare
Latest-write visibility	writer operation, later reader operation, expected value, actual value
Peer readable state	other cache valid/tag/data fields before and after the write window
Debug observability	debug pulse timing versus self-checking correctness result

8. Exercise C: Investigation Case C - Address Contract Failure

`make run-case-c`

- Observed symptom: a read returns a value associated with a different request history than the one expected by the trace
- Student task: determine which address bits are used for lookup and which address bits must be preserved to distinguish cache lines
- Interface contract: a cache lookup must preserve enough address information to distinguish different memory words
- **Student test extension:** create another trace with two addresses sharing index but using different tags
- **System lesson:** full-width address propagation is a correctness contract, not just a wiring detail

```
# Example aliasing pattern used by address_alias.trace
R 0 20 00000000
R 0 30 00000000
R 0 20 00000000
```

9. Exercise D: Investigation Case D - Same-Cycle Interaction

make run-case-d

- Observed symptom: a design can pass simple sequential tests but fail when two ports are active in the same cycle
- Student task: determine which same-cycle interaction changes the later visibility result
- Testing gap: purely sequential traces do not cover all multicore integration hazards
- **Stabilization method:** include same-cycle tests, assertions, and deterministic reproduction traces
- **Scalability warning:** four or eight cores multiply the number of possible simultaneous interactions

Sequential test	Concurrent test
C0 read, then C1 write, then C0 read	C0 read and C1 write in same cycle, then C0 read
easy to reason about order	requires defined visibility and invalidation timing
often passes weak test suites	exposes ordering, arbitration, or pulse-handling gaps

10. Exercise E: Write One New Trace

Each student adds one trace file in the traces directory. The trace should pass the baseline and expose at least one investigation case before patching.

Trace requirement	Example
At least five operations	R, R, W, R, W, R
At least two cores	Core 0 fills a line; Core 1 writes it
At least one repeated address	exposes read-after-write visibility
At least one same-index different-tag pair	exposes full-address contract errors
Short explanation	which invariant the trace targets

```
# Student-created trace example
R 0 28 00000000
R 1 28 00000000
W 1 28 ABCD0001
R 0 28 00000000
R 0 38 00000000
R 0 28 00000000
```

11. Exercise F: Choose One Provided Assertion Goal

Student instruction: choose one of the five predefined assertion goals below. Each goal is bounded, but the exact SystemVerilog implementation is not provided in the student handout. Start from the English invariant, identify the necessary signals, then implement a small check.

Choice	Assertion goal to implement; no exact RTL snippet is provided to students
F1	Core 0 must not accept a cache line as a hit if the stored line identity does not match the requested line identity. Suggested command: make run-case-c.
F2	Core 1 must not accept a cache line as a hit if the stored line identity does not match the requested line identity. Suggested command: make run-case-c or a student-created trace.
F3	After Core 0 completes a write to a line also held by Core 1, Core 1 must not continue to return a stale readable copy. Suggested command: make run-case-b.
F4	After Core 1 completes a write to a line also held by Core 0, Core 0 must not continue to return a stale readable copy. Suggested command: make run-case-b.
F5	A same-cycle interaction must not lose a visibility event required for the next read to observe the correct value. Suggested command: make run-case-d.

Where to place it: add the selected check near the end of rtl/two_core_cache_system.sv, before endmodule. Rebuild with make clean before rerunning the selected case command.

Submission requirement: report the selected assertion ID, the English invariant, the signals inspected, the SystemVerilog check attempted, the command run, and whether the check exposed the failure.

12. Generic Debugging Method When No Hint Is Available

Use this method before asking for the exact patch. The goal is to reconstruct the violated invariant from evidence, not to guess the solution from the exercise title.

Step 1: reproduce one failure and write down the first failing log line, including core, operation, address, got value, expected value, hit bit, and debug pulses.

Step 2: convert the first failure into an invariant such as latest-write visibility, correct address identity, or same-cycle event preservation.

Step 3: trace backward from returned data to hit/miss decision, cache valid/tag/data state, address decode, memory value, peer-cache update, and request timing.

Step 4: add temporary displays or a small assertion only after writing a hypothesis. Print only the signals needed to prove or disprove that hypothesis.

Step 5: patch one hypothesis at a time. A correct patch must keep make run-baseline passing and improve the selected failing case.

13. Gem5 Producer-Consumer Correlation Task

Goal: connect the RTL visibility bugs to a real multicore software pattern rather than only checking whether Gem5 launches.

```
make gem5-producer-consumer
```

What the script does: builds gem5/workloads/producer_consumer.c for RISC-V, then runs one process with two threads on two TimingSimpleCPU cores with private caches and shared L2.

Workload behavior: the created pthread acts as the producer and computes a deterministic checksum; the main thread acts as the consumer, waits on a shared ready flag, and verifies the checksum.

Expected workload output: PRODUCER role=producer, CONSUMER role=consumer, and PRODUCER_CONSUMER PASS.

Student task G1: open stats.txt and record simTicks, simInsts, and any visible per-core cycle or instruction counters reported by this Gem5 build.

Student task G2: explain why this workload makes visibility and ordering concrete at the architecture level, while the Verilator cases expose the lower-level RTL contracts that make such software patterns reliable.

Student task G3: explain why a passing Gem5 run does not prove the RTL implementation correct; Gem5 shows architecture-level behavior, while Verilator exposes the RTL contract violations.

Implementation note: this package uses exactly one created pthread plus the main thread. This avoids the earlier two-worker pthread failure and also avoids host-file mailbox operations that can trigger unsupported Gem5 SE syscalls.

RTL failure class	Architecture-level symptom	Gem5-style question
Lost visibility event	stale producer-consumer synchronization state	Could the consumer observe a flag/index update but not the associated data?
Wrong address identity	wrong shared variable or memory dependence behavior	Could two distinct shared variables or addresses be confused?
Same-cycle interaction failure	unstable behavior when producer and consumer contend	Which workload phases increase concurrent shared-line access?
Insufficient trace coverage	false confidence in scaling results	Which metrics cannot prove correctness by themselves?

14. Student Report Template

Each group submits a one-page report before the next session.

- Case selected: name one investigation case and the command used to reproduce it
- **Failing evidence:** quote the failing log line or describe the wrong read value
- Invariant: identify the broken system rule before claiming a root cause
- **New trace:** include the trace and explain why it targets the invariant
- New assertion: choose F1-F5, write the invariant, list the signals used, and state the observed pass/fail result
- **Architecture lesson: connect the RTL failure to the Gem5 producer-consumer pattern and explain how a visibility or ordering bug could mislead system-level evaluation**

15. Instructor Checkpoints and Rubric

Category	Excellent	Partial	Missing
Reproduction	runs baseline and at least two investigation cases with clear evidence	runs only one failing mode	cannot reproduce
Invariant explanation	states the violated system rule precisely	describes symptom only	no invariant
Trace quality	short, deterministic, targets one invariant	works but unfocused	no new trace
Assertion quality	checkable and tied to RTL state	English-only but correct	no assertion
Architecture connection	links RTL failure to multicore stabilization risk	generic discussion	no connection

16. Closing Discussion Questions

- **Correct module, broken system:** why can two correct cache modules fail when connected incorrectly?
- Invariant-first debugging: which case was easier to diagnose from the first failing log line, and why?
- **Trace versus random:** why should a verification plan include both directed traces and random stress?
- **Observability limit:** why are debug pulses useful but insufficient for correctness?
- **Scaling challenge:** how would this exercise become harder with four or eight cores?
- **Gem5 connection: why does the producer-consumer workload make visibility and ordering contracts more concrete, and why can it still not replace RTL verification?**
- **Open-source discipline:** why do reproducible commands, deterministic tests, and visible logs matter in prototype systems?

Appendix A: Files in the Zip Package

Path	Purpose
Makefile	single command entry point for all runs
rtl/two_core_cache_system.sv	multicore cache/memory RTL with selectable lab variants
rtl/tb_week11.sv	self-checking Verilator testbench
traces/coherence_directed.trace	directed trace for a repeated-address visibility case
traces/address_alias.trace	directed trace for an address-identity case
traces/simultaneous_race.trace	directed trace for a same-cycle interaction case
scripts/check_trace.py	Python golden trace checker
scripts/make_traces.py	regenerates trace inputs
gem5/run_producer_consumer.sh	builds and runs the two-core Gem5 producer-consumer workload
tests/expected_behavior.md	expected command-level behavior without solution hints
docs/debugging_without_hints.md	generic debugging workflow without solution hints
gem5/workloads/producer_consumer.c	two-thread producer-consumer source code
gem5/README_producer_consumer.md	Gem5 task notes and expected output

Appendix B: What Students Should Remember

- **Testing:** asks whether examples behave as expected
- **Verification:** asks whether properties remain true across many behaviors
- **Debugging:** finds the first point where the design violates an invariant
- **Stabilization:** turns an interesting prototype into a reproducible and trustworthy system
- **Multicore lesson:** shared state makes integration bugs visible only when another agent observes the stale or inconsistent state